- We have a complex problem which, internally, has a lot of intermediate results that can be reused frequently.
- To exploit this fact, we need a mechanism to remember the intermediate results so that we compute them only once (or at least as few times as possible).

That seems straightforward, right? For our example, it probably is: we have a very small number of possible entries (at most 100) and for each one we only store the sentiment computed. It seems likely that we would be ok with a very small amount of memory, to store our cache.

What do you think would happen, however, if instead we need to create a cache for the data gatherer in figure 7.2, storing all the messages across the major social networks mentioning a company, in their raw format? Consider there could be millions of those, for the “coolest” companies, and each one would on average require a few kilobytes (assuming we store media as links, or don’t store them at all). Even with just a hundred companies, it could be in the order of a few gigabytes.

Likewise, consider how an application like Facebook would work.

When you try to access your wall, an algorithm computes the best posts to show you, based on your friends’ walls, your preferences, the pages you follow etc…

It’s a pretty complex and resource-consuming algorithm, so you want to cache its results as much as possible: you probably don’t need to recompute the feed if the same user accesses her wall after one or five minutes (or maybe you